



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKA KAR

INFORMÁCIÓS RENDSZEREK TANSZÉK

Roll for Loyalty

Témavezető:

Dr.Gombos Gergő

egyetemi docens

Szerző:

Kecskés Norbert Dániel

programtervező informatikus BSc

Budapest, 2026

Tartalomjegyzék

1. Bevezetés	3
1.1. Motiváció, ötlet	3
1.2. Megvalósítás	4
2. Felhasználói dokumentáció	5
2.1. Célközönség	5
2.2. A program futtatása	5
2.3. Szerkezet és Használat	6
2.3.1. Singleplayer	6
2.3.2. Host Game	9
2.3.3. Join Game	9
2.3.4. Quit	10
2.3.5. Játéktér	11
2.3.6. Játék vége	22
3. Fejlesztői dokumentáció	24
3.1. Alkalmazott technológiák	24
3.2. Felhasználói esetek	25
3.2.1. Játék indítása	25
3.2.2. Játék kezdete	26
3.2.3. Egy forduló	26
3.2.4. Játékos köre	27
3.3. Szerkezeti áttekintés	27
3.4. Adatbázis réteg	27
3.4.1. Karakter adatbázis – egyjátékos mód	28
3.4.2. Játéktér adatbázis	31
3.5. Modell réteg	32
3.5.1. Egyjátékos modell réteg	32

3.5.2.	Többjátékos modell réteg	35
3.6.	Játéktér elemek	36
3.6.1.	Egyjátékos játéktér elemek	37
3.6.2.	Többjátékos játéktér elemek	39
3.7.	Tárgyrendszer	39
3.7.1.	Főbb osztályok:	40
3.7.2.	Új tárgy hozzáadása a rendszerhez	41
3.8.	Inventory rendszer	41
3.8.1.	Főbb osztályok:	42
3.9.	Felhasználói felület	43
3.9.1.	A főbb UI egységek:	43
3.9.2.	A felhasználói felület logikai alapjai:	43
3.9.3.	Támogatott interakciók:	44
3.10.	Tesztelés	44
3.10.1.	Attribútum- és ModifiableInt-tesztek	45
3.10.2.	Játékos- és bottesztek	45
3.10.3.	Tárgy- és inventory-tesztek	46
3.10.4.	Játéktér-tesztek	47
3.10.5.	Monster-tesztek	48
3.10.6.	A tesztek összefoglalása	48
3.11.	Továbbfejlesztési lehetőségek	49
4.	Összegzés	51
	Irodalomjegyzék	53
	Ábrajegyzék	54

1. fejezet

Bevezetés

A Roll for Loyalty egy körökre osztott, digitális asztali játék, amelynek alapját a felfedezés, a harc és a zsákmányszerzés hármasa alkotja. A játék során a résztvevők egyedül vagy más játékosokkal együtt egy ismeretlen, folyamatosan bővülő területet fedezhetnek fel, miközben különféle kihívásokkal, eseményekkel és ellenfelekkel találkoznak. A játék világában a kooperáció fontos szerepet játszhat, azonban nem minden szövetség tartós vagy megbízható. A játékmechanikák lehetőséget biztosítanak arra, hogy a szereplők stratégiai érdekeik szerint támogassák vagy akár elárulják egymást. Bár az együttműködés időnként elengedhetetlen lehet a túléléshez vagy a fejlődéshez, a végső cél egyértelmű: a győzelmet végül csak egyetlen játékos szerezheti meg.

1.1. Motiváció, ötlet

A játék alapelvei két ismert társasjáték, a Munchkin[1] és a Betrayal at House on the Hill[2] mechanikaiból merítenek inspirációt. A Munchkin elsősorban a karakterfejlődés, a tárgyrendszer, valamint a játékosok és a mesterséges intelligencia közötti interakciók kialakításában szolgált mintaként. Ezzel szemben a Betrayal at House on the Hill főként a játékterület moduláris, fokozatosan felfedezhető felépítésének megalkotásához adott alapötletet. A Roll for Loyalty ezekre az inspirációkra építve saját szabályrendszerrel rendelkező, körökre osztott játékot valósít meg, amelyben a játékosok különböző akciókat hajthatnak végre: mozoghatnak, új területeket fedezhetnek fel, szörnyekkel harcolhatnak, tárgyakat szerezhetnek, felszereléseket használhatnak, vagy akár stratégiai előnyök érdekében manipulálhatják a játék menetét.

A játék végső célja, hogy valamelyik játékos elsőként érje el a 10. szintet, ezzel bizonyítva rátermettségét, stratégiai fölényét és azt, hogy ő a legkiválóbb kalandor a kazamaták világában.

1.2. Megvalósítás

A játék fejlesztése a Unity[3] játékmotor segítségével valósult meg, amely modern fejlesztői környezetet biztosított a játék mechanikai, vizuális és hálózati rendszereinek kialakításához. A projekt a Unity 6000.3.6f1 verziójával készült, és elsődlegesen Windows platformon futtatható, önálló alkalmazásként. A program teljes egészében C# nyelven íródott. A fejlesztés során kiemelt figyelmet kapott a skálázhatóság és a bővíthetőség: az architektúra úgy lett kialakítva, hogy az egyjátékos és a többjátékos mód egyaránt jól elkülönített, mégis egymással kompatibilis rendszereken alapuljon. A hálózati funkcionalitás megvalósításához a Unity Netcode for GameObjects (NGO)[4] keretrendszer kerül alkalmazásra, amely kliens-szerver architektúrán keresztül biztosítja a játékosok közötti szinkronizációt.

2. fejezet

Felhasználói dokumentáció

2.1. Célközönség

A játék elsősorban azoknak szól, akik egyszerre rajonganak a mágikus világokért és a társasjátékokért. Különösen ajánlott azoknak, akik ismerik és szeretik a Munchkin vagy a House on the Hill típusú társasjátékokat, hiszen a játék szemléletmódja és mechanikái ezekhez állnak a legközelebb.

A Munchkin rajongói ismerős élményre lelhetnek a karakterosztályok és fajok rendszerében, a tárgyak és felszerelések gyűjtögetésében, valamint a többi játékkal való taktikai rivalizálásban, amely megengedi a csalást. Akik pedig a Betrayal at House on the Hill hangulatát kedvelik, azok számára vonzó lehet a felfedezés érzése, a véletlenszerűen alakuló helyzetek és az a bizonytalanság, hogy a következő lépés milyen fordulatot hoz.

A játék nem igényel mélyreható videójátékos tapasztalatot, így kezdő játékosok számára is könnyen befogadható, ugyanakkor a stratégiai döntések és a karakterépítés lehetőségei a tapasztaltabb játékosokat is le tudják kötni. Ideális választás, akár egyszemélynek, akár baráti köröknek, akik egy könnyed, mégis tartalmas mágikus élményre vágnak.

2.2. A program futtatása

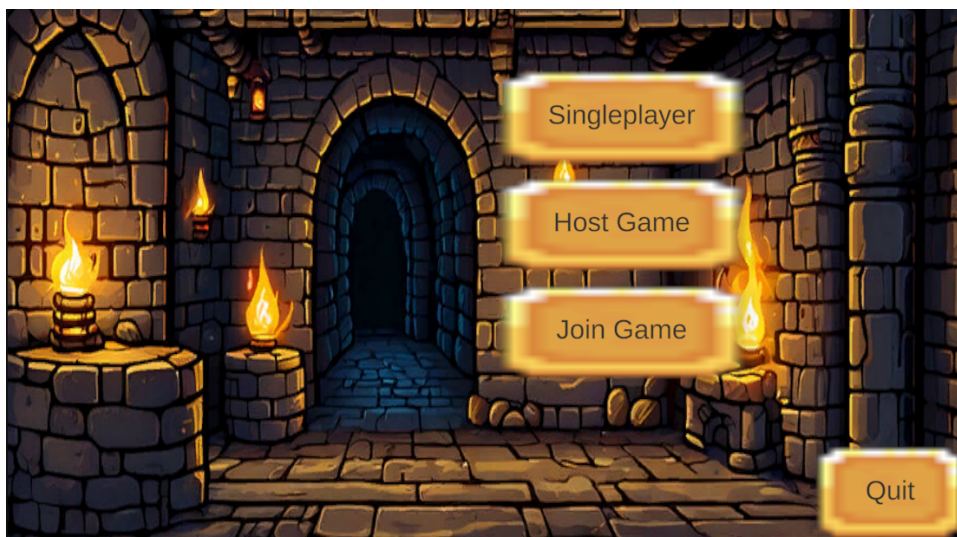
A Roll for Loyalty önálló alkalmazásként kerül kiadásra, így teljes mértékben lokálisan, telepítés nélkül futtatható. A játék támogatja a LAN-alapú többjátékos módot is, amelynek keretében a játékosok helyi hálózaton keresztül csatlakozhatnak

egymáshoz és közösen játszhatnak. Az alkalmazás elindításához elegendő a futtatható állomány (.exe) megnyitása, külső szerver vagy internetkapcsolat nem szükséges.

2.3. Szerkezet és Használat

A játék indítását követően a játékos a főmenübe kerül (lásd 2.1 ábra), ahol a bal egérgomb segítségével az alábbi lehetőségek közül választhat:

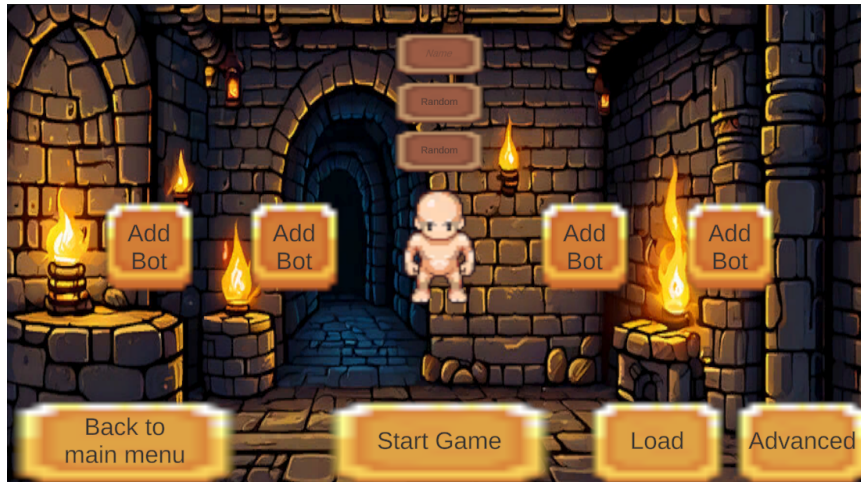
- Singleplayer – egyjátékos mód indítása
- Host Game – többjátékos játék létrehozása hosztként
- Join Game – csatlakozás egy meglévő többjátékos játékhoz kliensként
- Quit – kilépés az alkalmazásból



2.1. ábra. Főmenü

2.3.1. Singleplayer

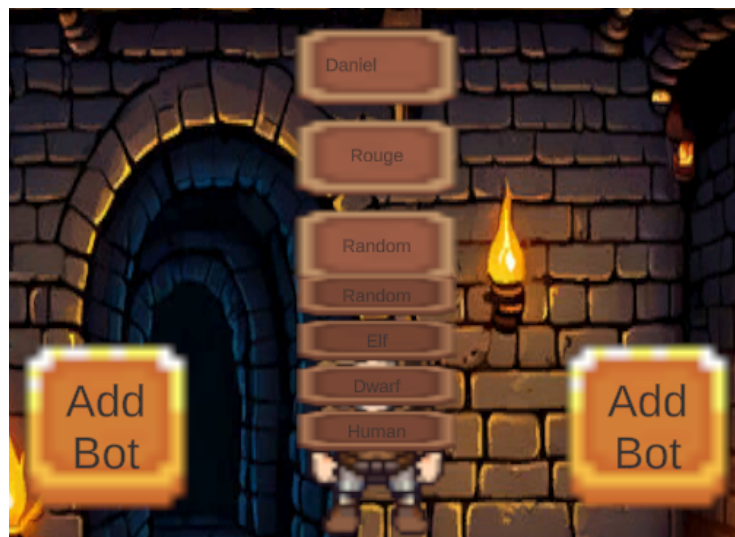
A *Singleplayer* gomb megnyomásával a játékos az egyjátékos módot választja ki. A program az egyjátékos menüre (*SinglePlayerMenu*) vált (lásd 2.2 ábra), ahol a játékos részletesen testre szabhatja a játékélményt.



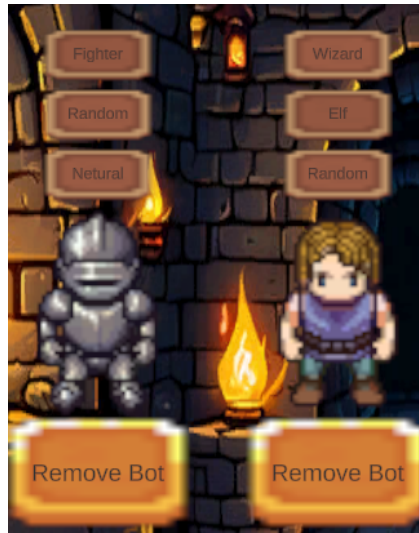
2.2. ábra. Egyjátékos menü

A testreszabási lehetőségek a következők:

- Karakter testreszabása (lásd 2.3 ábra)
- Mesterséges intelligencia által irányított karakterek hozzáadása (legfeljebb négy), amelyek egyenként testreszabhatóak (lásd 2.4 ábra)
- Botok eltávolítása igény szerint (lásd 2.4 ábra)

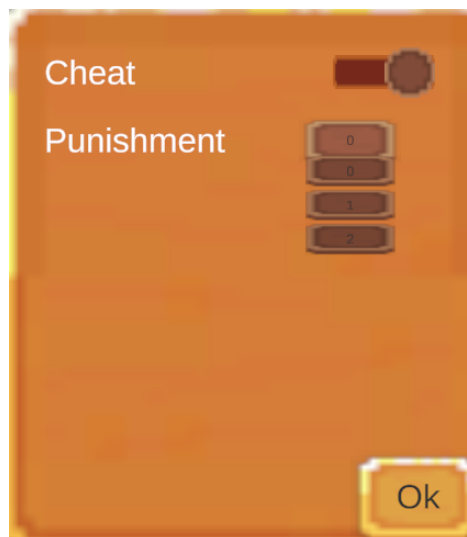


2.3. ábra. Játékos testre szabása



2.4. ábra. Bot testre szabása

A speciális beállítások (*Advanced Settings*) menüpontban további játékmechanikai opciók érhetők el (lásd 2.5 ábra): itt engedélyezhető a csalás funkció, és meghatározható a csalásért járó büntetés mértéke.

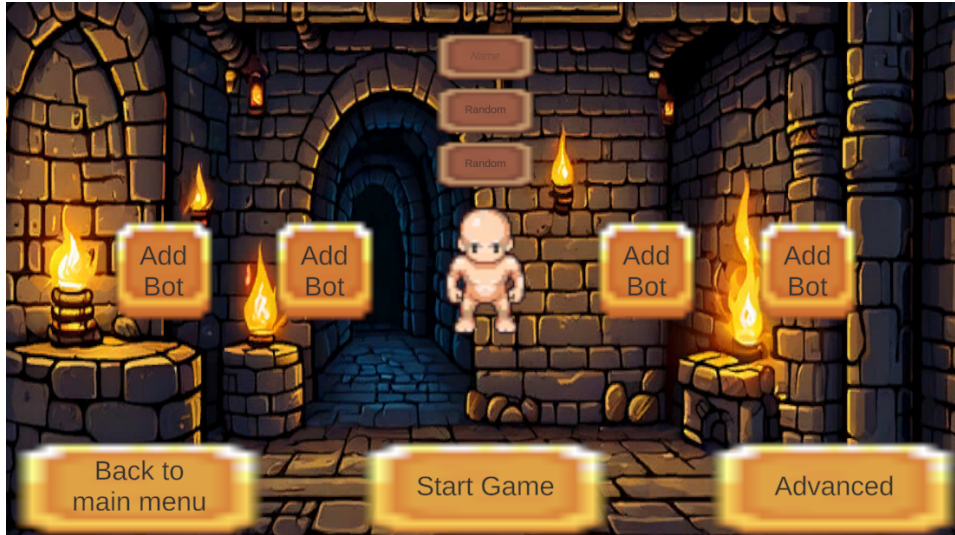


2.5. ábra. Speciális beállítások

A konfiguráció véglegesítése után a játék a *StartGame* gomb megnyomásával indítható el. Amennyiben a játékos rendelkezik korábban elmentett játékkal, lehetősége van annak betöltésére, így a játék egy korábbi állapotból folytatható. A *Back* gomb segítségével bármikor visszatérhet a főmenübe.

2.3.2. Host Game

A *Host Game* gomb megnyomásával a játékos hosztként lép be a többjátékos módba. A program a többjátékos hoszt menüre (*MultiplayerHostMenu*) vált (lásd 2.6 ábra), és ezzel egyidejűleg elindít egy szervert, amelyhez más játékosok helyi hálózaton keresztül csatlakozhatnak.

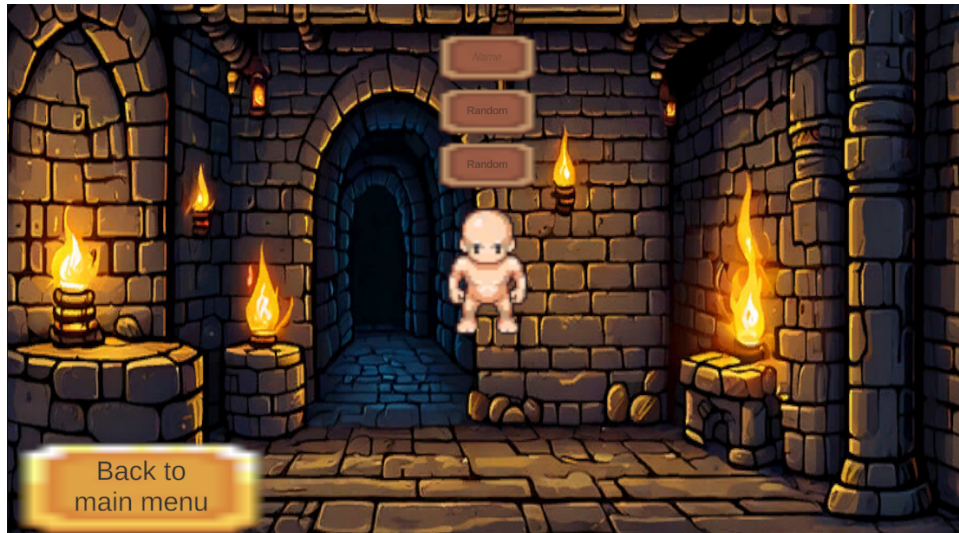


2.6. ábra. Többjátékos menü

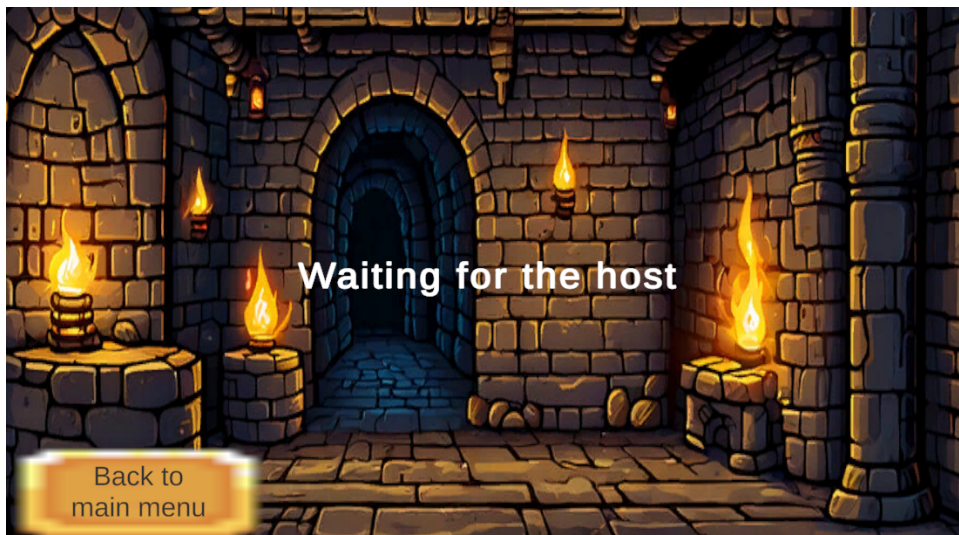
A hoszt az egyjátékos módhoz hasonlóan testreszabhatja a játékot: beállíthatja saját karakterét, hozzáadhat botokat, és módosíthatja a speciális beállításokat. A konfiguráció véglegesítése után a csatlakozott játékosok belépését követően a hoszt a *StartGame* gombbal indíthatja el a játékot. A *Back* gomb megnyomásakor a szerver leáll, és a csatlakozott kliensek kapcsolata megszakad.

2.3.3. Join Game

A *Join Game* gomb megnyomásával a játékos kliensként csatlakozik egy már futó többjátékos játékhoz. Ha elérhető aktív szerver található a helyi hálózaton, a csatlakozás automatikusan megtörténik (lásd 2.7 ábra). Ha nem, a játékos egy várakozó felületre kerül (lásd 2.8 ábra), ahol addig maradhat, amíg egy hoszt el nem indítja a szervert, vagy bármikor visszaléphet a főmenübe.



2.7. ábra. Többjátékos menü



2.8. ábra. Többjátékos Várakozó menü

A kliens kizárólag saját karakterét tudja testre szabni, a globális játékbeállításokat kizárólag a hoszt módosíthatja. Miután a hoszt elindítja a játékot, a kliens automatikusan a közös játékba kerül.

2.3.4. Quit

A *Quit* gomb megnyomásával a játékos bezárja az alkalmazást.

2.3.5. Játéktér

A játék elindítását követően – a választott játékmódtól függően – az egyjátékos (*SingleplayerGameField*) vagy a többjátékos (*MultiplayerGameField*) játéktér töltődik be (lásd 2.9 ábra). A két játéktér felépítése alapvetően megegyezik, azonban többjátékos módban a többi résztvevő karaktere is megjelenik a közös játéktérületen.



2.9. ábra. Játéktér

A játéktér két fő részre osztható

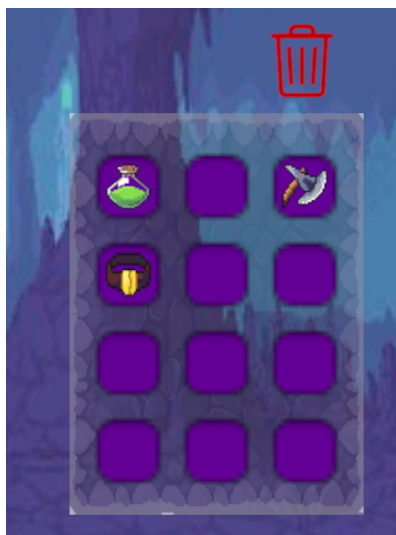
- A játékoshoz tartozó interakciós felület
- A dinamikusan bővülő játéktérület

Játékoshoz tartozó kezelőfelület

A kezelőfelület célja, hogy a játékos számára minden fontos információ könnyen áttekinthető és gyorsan elérhető legyen, ezáltal támogatva a stratégiai tervezést, a karakterfejlesztést és a játékélmény folyamatosságát.

A kezelőfelület elemei:

- Inventory – a játékos tárgyait tartalmazó tárolófelület (lásd 2.10 ábra)



2.10. ábra. Inventory

- Equipment inventory – az aktuálisan felszerelt tárgyak megjelenítése (lásd 2.11 ábra)



2.11. ábra. Equipment inventory

- Statisztikák listája – a karakter aktuális jellemzői (lásd 2.12 ábra)

Wizard	Dwarf	1	6	10	10	10	10	10	5	4
Class	Race	Level	Health	Stregth	Agility	Intelect	Charisma	Attack Power	Movement	Draws

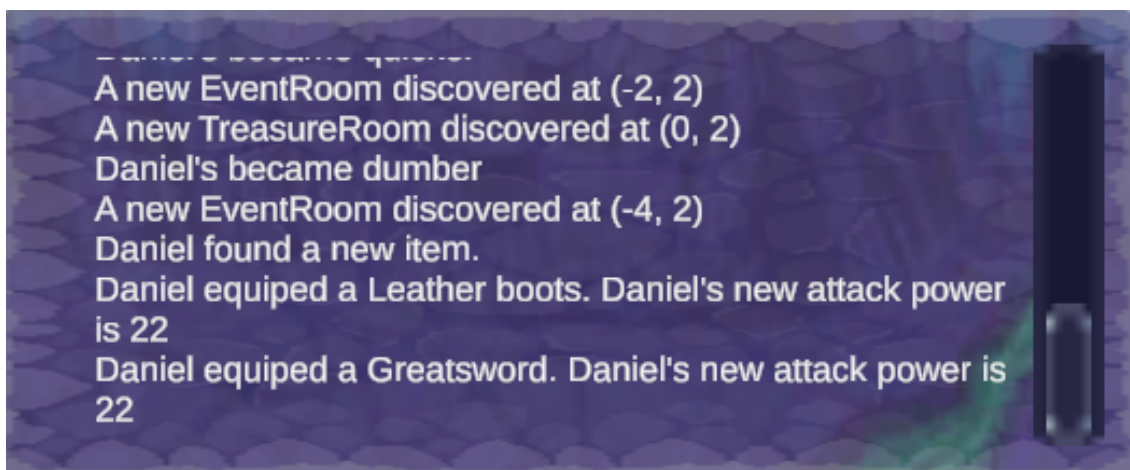
2.12. ábra. Statisztikák listája

- Vezérlőgombok (lásd 2.13 ábra)



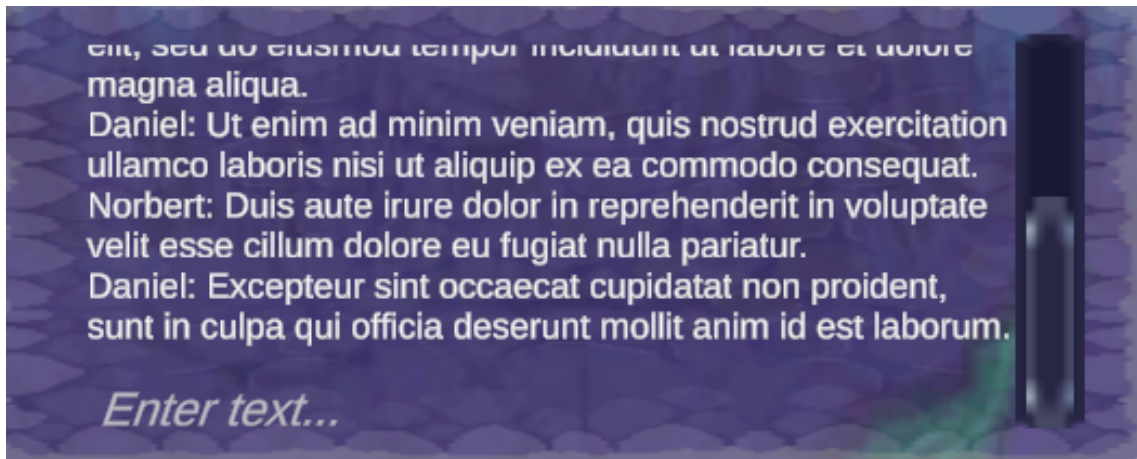
2.13. ábra. Vezérlőgombok

- játék befejezése
 - játék mentése (csak egyjátékos módban)
 - tárgy húzása
 - kör befejezése
 - csaló játékos megjelölése (ha engedélyezett)
- Eseménynapló (*Log*) – az összes fontosabb esemény szöveges megjelenítése (lásd 2.14 ábra)



2.14. ábra. Eseménynapló

- Kommunikációnapló (*Chat*) - a játékosok kommunikációjának megjelenítése (csak többjátékos módban) (lásd 2.15 ábra)



2.15. ábra. Kommunikációs felület

Játékterület

A játékterület központi eleme a dinamikusan épülő pálya, amely fokozatosan bővül a játék során, ahogy a játékosok új területeket fedeznek fel. A pályán a karakterek különböző eseményekkel, szörnyekkel, csapdákkal, kincsekkel és egyéb interaktív objektumokkal találkoznak.

A játékterület elemei:

- *Plus* objektumok – jelölik az újonnan felfedezhető területeket
- Szobák:
 - üres szoba
 - folyosó
 - kincsszoba
 - eseményszoba
 - szörnyszoba
- Játékos karakter(ek)
- Bot karakter(ek)

A karakterek mozgatása kattintással történik: a játékos a rendelkezésre álló szobákra vagy Plus objektumokra kattinthat. Ha ismert szobára kattint, karaktere egyszerűen belép oda, ha Plus objektumra kattint, karaktere odalép, majd egy új szoba kerül felfedésre. A pálya fokozatos bővülése minden játékalalmat egyedivé tesz.

Játékos

A játékos a játék központi szereplője, akinek fejlődése, döntései és stratégiája közvetlenül befolyásolják a játék kimenetelét. Minden karakter több alapvető tulajdonsággal és állapottal rendelkezik.

Főbb jellemzők:

- Név
- Faj
- Kaszt
- Szint
- Attribútumok:
 - Életerő
 - Erő
 - Ügyesség
 - Intelligencia
 - Karizma
 - Állóképesség
- Támadási erő
- Mozgáspontok
- Húzható lapok száma
- Csalási állapot (csalt-e az aktuális körben)

A játékos köre

A játék körökre osztva működik. Minden kör elején a játékos mozgáspontjai frissülnek, és a csalási állapot visszaáll. Egy kör alapvetően az alábbi lépésekből állhat:

- Mozgás a pályán a rendelkezésre álló mozgáspontok erejéig

- Új területek felfedezése
- Kincs húzása
- A kör befejezése

Ha a játékos új szobát fedez fel, mozgása általában azonnal nullázódik, kivéve ha a felfedezett terület üres szoba vagy folyosó.

Bármikor végrehajtható műveletek

Bizonyos műveletek nem kizárólag a játékos saját köréhez kötődnek, hanem a játék során bármikor végrehajthatók:

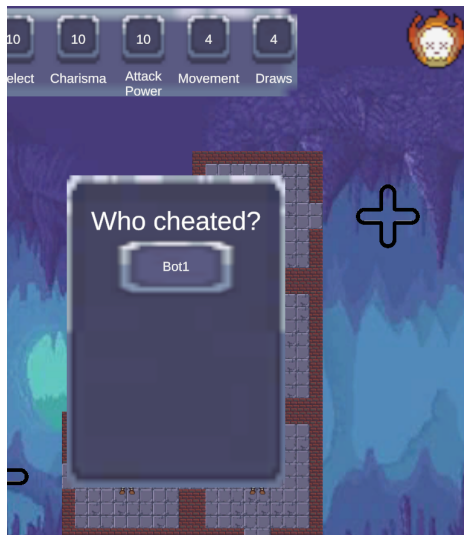
- A játék mentése (ha a játékmód lehetővé teszi)
- Az inventory rendezése
- Tárgyak felszedése vagy lerakása
- Csaló játékos megjelölése (ha a funkció engedélyezett)
- Beavatkozás más játékos vagy bot harcába

Csalás mechanika

Ha a speciális beállítások között a csalás engedélyezett, a játékos szabályszegő előnyökhöz juthat:

- Megszűnik a mozgáskorlátozás
- Megszűnik a húzási korlátozás
- A játékos szabadon helyezhet tárgyakat az equipment inventoryba a normál szabályoktól függetlenül.

A csalás azonban kockázatos: a többi játékos és a botok egyaránt lehetőséget kapnak a csaló megjelölésére. Sikeres leleplezés esetén a beállított büntetés lép életbe, amely jellemzően a tárgyak részleges elvesztésével jár (lásd 2.16 ábra).



2.16. ábra. Csaló megjelölése

Mesterséges intelligencia

A mesterséges intelligencia által irányított karakterek (botok) alapjaiban hasonlóan működnek, mint a játékosok, azonban döntéseiket automatizált viselkedési rendszer irányítja.

A botok nyomon követik a többi karakterhez fűződő viszonyukat egy reputációs rendszer segítségével. A reputáció pozitív vagy negatív irányban változhat az alábbi eseményektől:

- Egy karakter segíti a harc során → pozitív hatás
- Egy karakter hátráltatja a harc botot → negatív hatás
- Egy karakter csaláson kapja vagy megjelöli a botot → negatív hatás

A bot köre

A bot körének főbb lépései automatikusan zajlanak le:

- Mozgás a maximálisan elérhető távolságig
- Tárgy húzása, ha lehetséges
- Döntés a megszerzett tárgyról: felszerelés, megtartás vagy eldobás
- Esetleges csaló játékos feljelentése reputációs logika alapján
- Saját stratégiai döntés alapján akár maga is csalni próbálhat

- Kör befejezése

A botok csalási lehetőségei a játékosokéhoz hasonló szabályok szerint működnek, így amennyiben a csalás engedélyezett, a mesterséges intelligencia is élhet a szabályszegés nyújtotta előnyökkel. Ugyanakkor a botok is lebuktathatók, amely negatív következményekkel járhat számukra.

Körön kívüli bot interakciók

A botok körükön kívül is képesek döntéseket hozni, például szörnyharcba való beavatkozáskor. A bot reputációs rendszerétől és stratégiai helyzetétől függően dönthet a másik karaktert:

- Támogatja
- Hátráltatja
- Semlegessen viselkedik

Tárgyak

A tárgyak a játék egyik meghatározó elemét alkotják, mivel a karakterfejlődés és megerősödés legfontosabb eszközei. A különböző tárgyak segítségével a játékos növelheti attribútumait, és ideiglenes vagy állandó bónuszokra tehet szert.

Tárgykategóriák:

- Főkézben viselhető fegyver
- Másodkézben használható fegyver
- Pajzs
- Sisak
- Páncél
- Lábvért
- Kiegészítők
- Bájitalok

- Negatív hatású tárgyak
- Alapértelmezett

A felszerelhető tárgyak jellemzően állandó bónuszokat biztosítanak, míg a bájitalok és negatív hatású tárgyak ideiglenes hatással rendelkeznek. Bizonyos tárgyakból – különösen a bájitaloktól és negatív hatású tárgyaktól – egy inventory slotban több darab is tárolható egyszerre, ilyenkor a tárgy ikonjának jobb alsó sarkában a mennyiség is megjelenik.

Tárgyak kezelése

A tárgykezelő rendszer lehetővé teszi az inventory rugalmas kezelését. A tárgyak mozgatása bal egérgombbal, húzás-ejtés módszerrel történik.

Végrehajtható műveletek:

- Tárgyak áthelyezése ugyanazon inventory különböző slotjai között
- Két tárgy helyének felcserélése (lásd 2.17 ábra)



2.17. ábra. Tárgyak cseréje

- Tárgyak mozgatása az általános inventory és az equipment inventory között (lásd 2.18 ábra)



2.18. ábra. Tárgy felszerelése

- Tárgyak törlése a kuka ikonra húzással (lásd 2.19 ábra)



2.19. ábra. Tárgy törlése

Ha a játékos egy tárgyat az equipment inventoryba helyez, annak bónuszai automatikusan hozzáadódnak a karakter megfelelő attribútumához. Az eltávolításkor a bónuszok megszűnnek. A bájital és negatív hatású tárgyak kattintással aktiválhatók, és egy körre módosítják a karakter tulajdonságait.

Szobák

A játéktérület felfedezése során a játékos különböző típusú szobákkal találkozhat, amelyek eltérő eseményeket, lehetőségeket vagy veszélyeket rejtenek.

- **Üres szoba** – nem történik különleges esemény, a pályán való haladást szolgálja
- **Folyosó** – az üres szobához hasonló, de vizuálisan szűkebb átjáró, amely összeköti a pálya részeit
- **Kincsszoba** – a játékos egy tárgyat húzhat a kincspakliból.
- **Eseményszoba** – véletlenszerű pozitív vagy negatív eseményt vált ki:
 - Köd
 - Teleportálás
 - Faj- vagy kasztváltás
 - Szintváltozás
 - Attribútummódosítás
- **Szörnyszoba** – a játékos egy szörnnyel találkozik, amelyet le kell győznie

Harc

A harc megkezdése előtt egy 5 másodperces visszaszámlálás indul (lásd 2.20 ábra). Ezalatt a többi játékos és bot eldöntheti, hogy beavatkoznak-e a küzdelembe, a harcoló fél támogatásával vagy hátráltatásával. Az időzítő lejártával a harc automatikusan lezajlik: az a fél győz, amelynek végső ereje magasabb, döntetlen esetén a szörny nyer.



2.20. ábra. Karakter és szörny harcolása

Harc kimenetele:

- **Játékos győzelme**

- Szintlépés
- Kincs(ek) húzása

- **Szörny győzelme** Egy negatív következmény:

- Fajváltás
- Kasztváltás
- Szintcsökkenés
- Attribútum csökkenés

A szörnyek négy nehézségi kategóriába sorolhatók:

- Könnyű
- Közepes
- Nehéz
- Halálos

2.3.6. Játék vége

A játék kétféleképpen érhet véget:

- A játékos kilép
- Valamelyik karakter teljesíti a győzelmi feltételt

A győztes az a játékos vagy bot, aki elsőként éri el a 10. szintet szörny legyőzésével. Fontos, hogy az eseményszoba által adott szintlépések nem számítanak be a győzelmi feltételbe.

Győzelem esetén a játéktér helyét egy nyertes képernyő (*Win*) veszi át, amely megjeleníti a győztes nevét. Többjátékos módban a kliensek automatikusan lecsatlakoznak, a hoszt leállítja a szerver, és minden résztvevőnél megjelenik a nyertes képernyő (lásd 2.21 ábra).



2.21. ábra. Nyertes képernyő

Ha a játékos saját döntése alapján kilép, a játékmenet megszakad és a program visszatér a főmenübe. Többjátékos módban kliens kilépésekor a karakter irányítását egy bot veszi át, hoszt kilépésekor a szerver leáll és minden kliens kapcsolata megszakad.

3. fejezet

Fejlesztői dokumentáció

3.1. Alkalmazott technológiák

A *Roll for Loyalty* fejlesztése során a program teljes egészében a Unity játékmotorra épül, amely biztosítja a szerver oldali logikát, a kliens oldali felhasználói felületet, valamint a grafikai és hálózati rendszerek egységes kezelését. A fejlesztés nyelve a C#.

A felhasználói felület kialakítása a Unity beépített UI rendszerével történt: Canvas alapú megjelenítés, TextMeshPro szövegkezelés és EventSystem alapú interakciókezelés segítségével. Ez biztosítja a menürendszer, az inventory kezelés, a karakterkezelő felületek és a játékközbéli vezérlőelemek működését.

A többjátékos funkcionalitást a Unity Netcode for GameObjects (NGO)[4] keretrendszer valósítja meg, amely kliens-szerver architektúrán keresztül biztosítja a játékosok közötti szinkronizációt. A hálózati kommunikációt RPC hívások, NetworkVariable adattagok és a Unity Transport réteg támogatja, lehetővé téve a helyi hálózaton történő többjátékos kapcsolódást.

Az adatkezelés és mentési rendszer a C# fájlkezelési lehetőségeire, elsősorban a System.IO könyvtárra épül, amely lehetővé teszi a játékállapot, a karakteradatok, az inventory tartalmak és a pályastruktúrák lokális tárolását és visszatöltését.

A vizuális elemek prefab alapú objektumkezeléssel, sprite rendszerekkel és a Unity Resources.Load() erőforrás-betöltési megoldásával készültek. A fejlesztési környezet a Microsoft Visual Studio IDE volt, a verziókövetés Git alapú rendszerrel valósul meg.

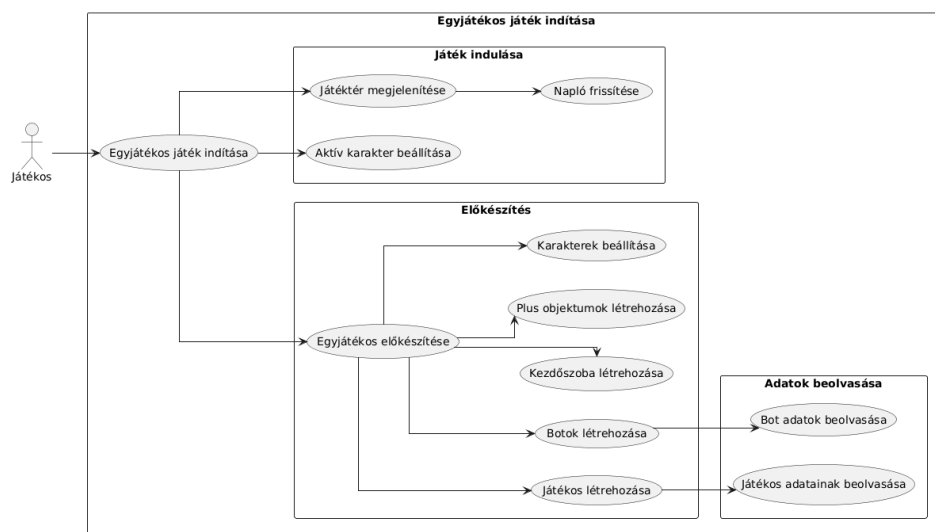
3.2. Felhasználói esetek

3.2.1. Játék indítása

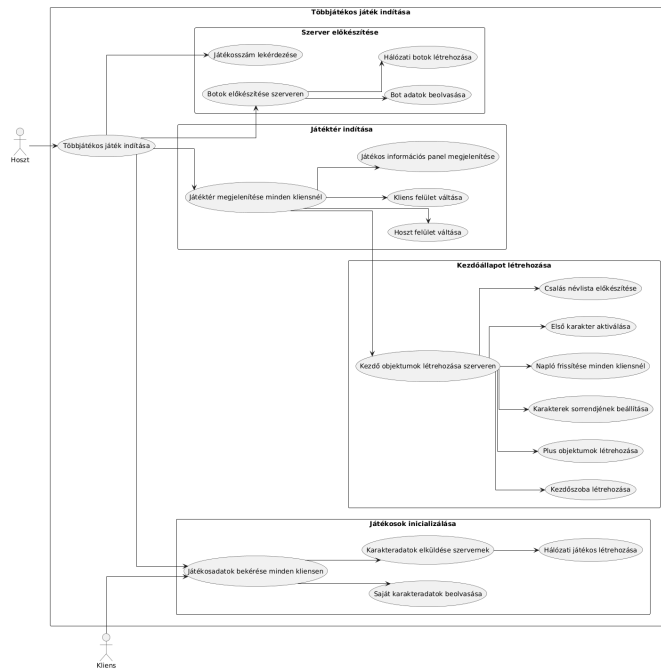
A játék indítása a kiválasztott játékmódtól függően történik.

- Egyjátékos mód esetén a játékos (lásd 3.1 ábra)
- Többjátékos mód esetén (lásd 3.2 ábra) a hoszt

felelős az indításért, miután a szükséges beállításokat véglegesítette, és minden résztvevő sikeresen csatlakozott.



3.1. ábra. Az egyjátékos mód indításának folyamata



3.2. ábra. A többjátékos mód indításának folyamata

Az indításkor a rendszer először létrehozza a szükséges adatstruktúrákat:

- A karakteradatbázist (minden játékos és bot adatával)
- A játéktér adatbázisát (a pálya összes aktív elemével)

Ezt követően inicializálja a felhasználói nézetet és megjeleníti:

- Karaktereket
- Botokat
- A kezdeti pályaelemeket

3.2.2. Játék kezdete

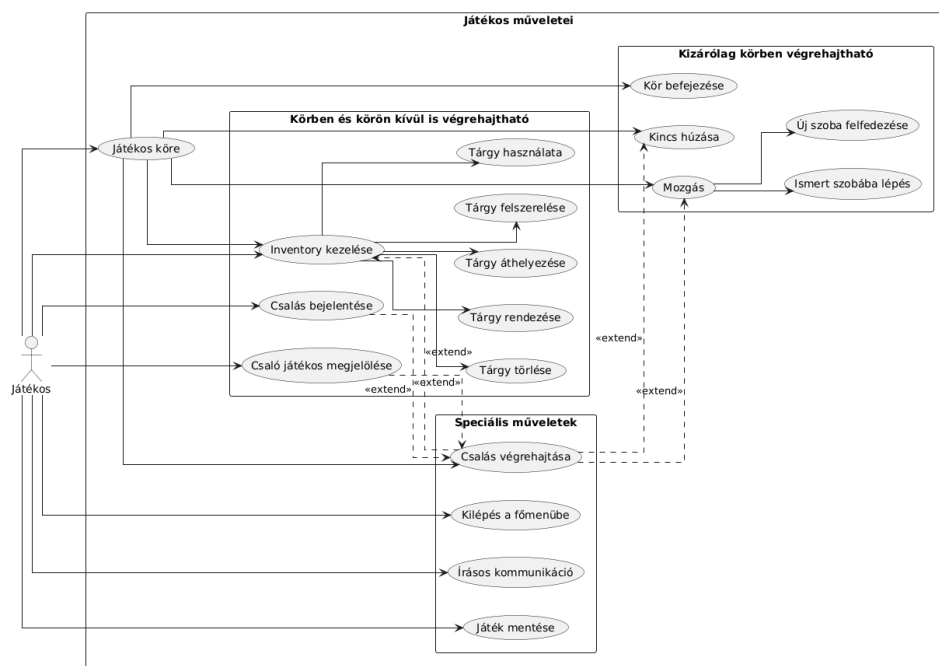
A játék indulásakor a rendszer véletlenszerűen összekeveri a karakterek listáját, ezzel meghatározva a körök sorrendjét. A sorrend meghatározása után a rendszer aktiválja az első karaktert és elindítja annak körét.

3.2.3. Egy forduló

Egy forduló során a játékosok egymás után végrehajtják saját köreiket. Az utolsó játékos körét követően a lista eleje következik – ezzel új forduló indul. Ha valamelyik karakter eléri a 10. szintet, a jelenlegi forduló megszakad és a játék véget ér.

3.2.4. Játékos köre

A játékos saját köre során öt alapvető akciótípus közül választhat, amelyeket a lehetőségeihez mérten tetszőleges mennyiségben hajthat végre (lásd 3.3 ábra). Ha a csalás mechanika engedélyezett, bizonyos műveleteket normál esetben tiltott módon is ismételhet, ugyanakkor ez lehetőséget teremt a többi játékos és bot számára a leleplezésre.



3.3. ábra. A játékos által végrehajtható műveletek

3.3. Szerkezeti áttekintés

A program felépítése osztályalapú architektúrára épül. A rendszer különböző osztályokból áll, amelyek egy része a játék működéséhez szükséges logikát valósítja meg, míg más osztályok adatbázis jellegű szerepet töltenek be – azaz strukturált formában tárolják és kezelik a játék állapotát.

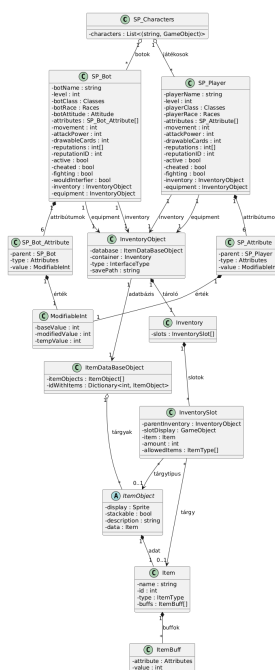
3.4. Adatbázis réteg

Az adatbázis réteg feladata a játékállapot teljes körű tárolása: minden olyan információ, amely egy adott pillanatban szükséges a játék pontos leírásához. Ide tartoznak:

- Ez a réteg biztosítja, hogy a játék bármely állapota lekérdezhető, módosítható, menthető és visszatölthető legyen.

3.4.1. Karakter adatbázis – egyjátékos mód

A karakter adatbázis minden olyan információt tárol, amely a játékban szereplő karakterek teljes leírásához szükséges, beleértve a játékosokat és a botokat egyaránt (lásd 3.4 ábra).



3.4. ábra. Egyjátékos karakter adatbázis szerkezete

A legfontosabb táblák:

- **SP_Characters** – a karakterek sorrendjét tároló lista tartalmazza:
 - A karakter nevét (*Name*)
 - A GameObject referenciáját (*character*)
- **SP_Player** – a játékos karakter adatai:
 - Név (*PlayerName*)

- Szint (*Level*)
- Kaszt (*PlayerClass*)
- Faj (*PlayerRace*)
- Attribútumok (*Attributs*)
- Mozgáspont (*Movement*)
- Támadóerő (*AttackPower*)
- Húzható kártyák száma (*DrawableCards*)
- Reputációs értékek (*Reputations*)
- Aktív és harci állapot (*ACtive, Fighting*)
- Inventory
- Equipment
- **SP_Bot** – a botok adatai, az SP_Player-hez hasonló struktúrával, kiegészítve
 - Viselkedési típus (*botAttitude*)
 - Beavatkozási hajlandósággal (*wouldInterfier*)
- **SP_Attribute / SP_Bot_Attribute** – egy karakter egyes attribútumait leíró rekordok:
 - Szülőkarakter (*parent*)
 - Típus (*type*)
 - Érték (*value*)
- **InventoryObject** – egy inventory típusú adattároló (általános vagy felszerelés), mentési útvonallal
- **InventorySlot** – egy inventory mezőt leíró rekord:
 - Szülőinventory (*parentInventory*)
 - Megjelenítő elem (*slotDisplay*)
 - Tárgy (*item*)
 - Mennyiség (*amount*)
 - Engedélyezett tárgytípusok (*allowedItems*)

- **ItemDataBaseObject** – a játékban elérhető összes tárgy adatbázisa, azonosító alapú gyors elérést biztosítva
- **ItemObject** – egy tárgy alapadatai ScriptableObject formában:
 - Kép (*display*)
 - Halmazhatóság (*stackable*)
 - Leírás (*description*)
 - Adat (*data*)
- **Item** – egy konkrét tárgy részletes adatai:
 - Név (*name*)
 - Azonosító (*id*)
 - Típus (*type*)
 - Buffok (*buffs*)
- **ItemBuff** – egy tárgy által adott attribútummódosító:
 - Módosított attribútum (*attribute*)
 - Érték (*value*)
- **ModifiableInt**
 - Alap érték (*baseValue*)
 - Ideiglenes érték (*tempValue*)
 - Módosított érték (*modifiedValue*)

Karakter adatbázis – többjátékos mód

A többjátékos karakter adatbázis felépítése nagyrészt megegyezik az egyjátékos változattal, azonban a hálózati működés miatt néhány fontos eltérést tartalmaz. A kritikus adatok nem egyszerű lokális változóként, hanem hálózaton szinkronizált NetworkVariable értékként tárolódnak (lásd 3.5 ábra).



- ### 3.4.2. Játéktér adatbázis

31

a játéktéren található objektumok pozíció alapján tárolhatók, lekérdezhetők, módosíthatók vagy eltávolíthatók legyenek, mindkét játékmódban. (lásd 3.6 ábra).



3.6. ábra. Játéktér adatbázis szerkezete

3.5. Modell réteg

A modell réteg a program működéséhez szükséges játékmeneti logikát megvalósító osztályok összessége. Ezek az osztályok az adatbázis rétegben tárolt aktuális állapot alapján működnek, majd a játék eseményeinek megfelelően módosítják azokat.

A modell réteg feladatai:

- A játékosok és botok műveleteinek feldolgozása
- A játékszabályok alkalmazása és az egyes műveletek feltételeinek ellenőrzése
- A karakterek, játéktér és tárgyak állapotának módosítása
- A harc, mozgás, tárgyhasználat és egyéb események kezelése
- A változások továbbítása a megjelenítésért felelős osztályok felé
- Eseménynaplózás biztosítása a Log rendszer számára

3.5.1. Egyjátékos modell réteg

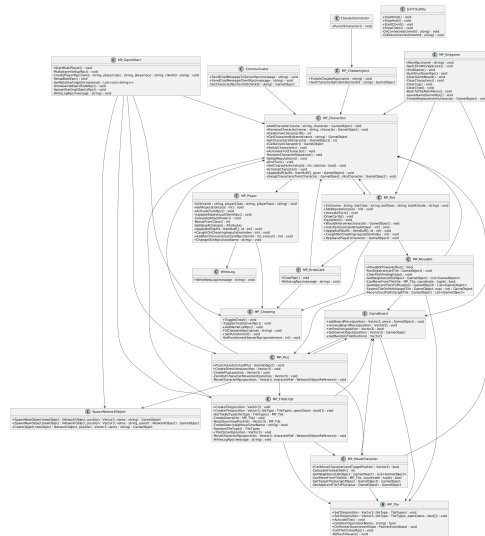
Az egyjátékos modell réteg tartalmazza azokat az osztályokat, amelyek a játék működését, szabályait és eseményeit valósítják meg (lásd 3.7 ábra).

- *CanMoveCharacter*: eldönti, hogy a karakter el tud-e jutni a kiválasztott célpozícióra.
- *CalculateFastestPath*: kiszámítja a célhoz vezető legkisebb mozgásköltségű útvonalat.
- *GetNeighbors*: lekéri az adott mező szomszédos mezőit.
- *CanMoveFromTile*: ellenőrzi, hogy egy adott irányba van-e nyitott ajtó.
- *MoveCharacterToTargetPosition*: áthelyezi a karaktert a célpozícióra.
- **SP_MoveBot** – a botok automatikus mozgása a legközelebbi elérhető Plusz objektum felé
 - *MoveBotTowardsPlus*: elindítja a bot mozgását a kiválasztott plusz objektum irányába.
 - *RunDijkstra*: kiszámítja az elérhető mezők mozgásköltségét.
 - *GetNeighbors*: lekéri az aktuális mező szomszédait.
 - *GetAdjacentTilesToPlus*: megkeresi a plusz objektum melletti mezőket.
 - *FastestTileOnPath*: kiválasztja azt a mezőt, ameddig a bot az aktuális mozgáspontjaival el tud jutni.
- **SP_TileScript** – mezők létrehozása és inicializálása: típus, ajtók, megjelenítés
 - *CreateTile*: létrehoz egy új mezőt a megadott pozíción.
 - *GetTileByType*: kiválasztja a mezőtípushoz tartozó prefab objektumot.
 - *CreateDoors*: létrehozza vagy beállítja a mező ajtóit.
 - *RandomTileType*: véletlenszerű mezőtípust választ.
 - *TileClicked*: kezeli a mezőre történő kattintást.
- **SP_Plus** – a játéktér bővítési pontjainak kezelése
 - *PlusClicked*: kezeli a plusz objektumra történő kattintást.
 - *CreateDirections*: létrehozza az új mező körüli lehetséges plusz objektumokat.
 - *CreatePlus*: létrehoz egy plusz objektumot a megadott pozíción.

- *NullCharacterMovement*: bizonyos mezőtípusok esetén lenullázza az aktív karakter mozgását.
- **SP_DrawCard** – a tárgyhúzás logikája
 - *Draw*: az aktív karakter számára új tárgyat húz.
- **SP_Cheating / SP_CheateingList** – a csalás engedélyezése, büntetés beállítása, felhasználói választások kezelése
 - *ToggleCheat*: be- vagy kikapcsolja a csalás lehetőségét.
 - *SetPunishment*: beállítja a csalásért járó büntetés mértékét.
 - *FillDropdown*: feltölti a választólistát a karakterek neveivel.
 - *PunishCheater*: megbünteti a kiválasztott csaló karaktert, ha valóban csalt.
- **SaveLoad** – játékállapot mentése és visszatöltése.
 - *SaveFile*: elmenti a játék aktuális állapotát.
 - *CreateGameBoardList*: előállítja a játéktér mentéséhez szükséges adatokat.
 - *CreatePlayerList*: előállítja a játékos mentéséhez szükséges adatokat.
 - *CreateBotList*: előállítja a bot mentéséhez szükséges adatokat.
 - *LoadFile*: betölti a korábban elmentett játékállapotot.

3.5.2. Többjátékos modell réteg

A többjátékos modell réteg felépítése nagyrészt megegyezik az egyjátékos modell réteggel, azonban a hálózati működés miatt több osztály kiegészül a Unity Netcode rendszer elemeivel (lásd 3.8 ábra).



3.8. ábra. Többjátékos modell réteg szerkezete

A többjátékos modell réteg fő eltérései:

- Az osztályok jelentős része *NetworkBehaviour*-ből származik
- Az adatok *NetworkVariable* vagy *NetworkList* formában tárolódnak
- A műveletek egy része *RPC* hívásokkal fut le a szerveren vagy a klienseken
- Az objektumok létrehozása és törlése *NetworkObject* segítségével történik
- A karaktermozgatás, kártyahúzás, naplózás és mezőkezelés hálózaton szinkronizált
- Minden kliens karakteréhez kliensazonosító tartozik

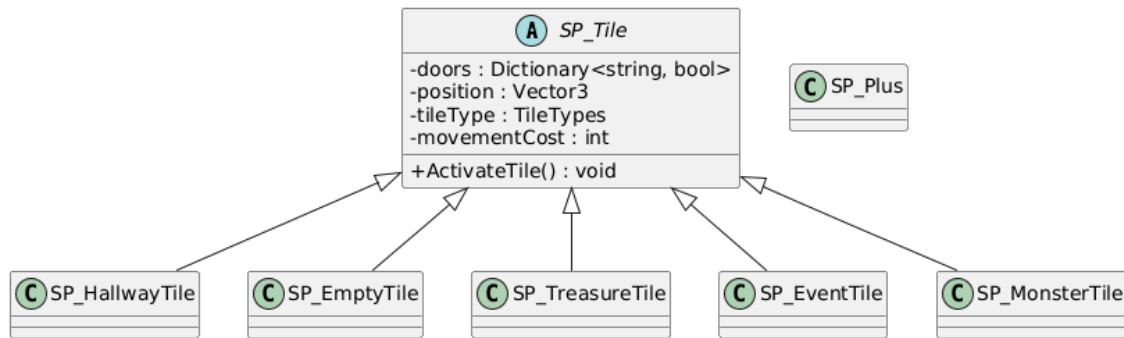
Új elemként jelenik meg a többjátékos rétegben:

- **JoinToLobby** - hoszt és kliens csatlakozás, kilépés és lobby állapot kezelése
- **Communicator** - játékosok közötti üzenetküldés *RPC* hívásokon keresztül

3.6. Játéktér elemek

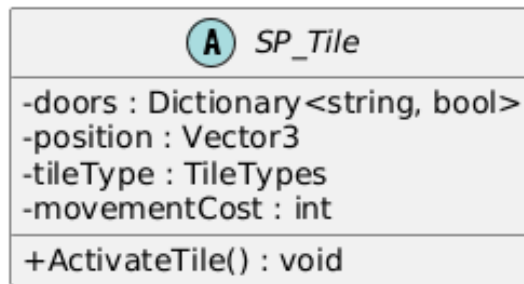
A játéktér elemei azok az objektumok, amelyekből a pálya felépül. Az egyjátékos változat absztrakt *SP_Tile* alaposztályra épül, a többjátékos változat ennek hálózati megfelelője, az *MP_Tile* (lásd 3.9 ábra).

3.6.1. Egyjátékos játéktér elemek



3.9. ábra. Játéktér elemei

Az SP_Tile (lásd 3.10 ábra) leszármazottai:



3.10. ábra. Abstract szoba

- SP_HallwayTile - folyosó, összekötő elem, nem vált ki eseményt
- SP_EmptyTile - üres szoba, nem vált ki eseményt
- SP_TreasureTile - kincsszoba, aktiválásakor az aktív karakter egy lapot húzhat
- SP_EventTile - eseményszoba, véletlenszerű pozitív vagy negatív eseményt hajt végre (lásd 3.11. ábra)



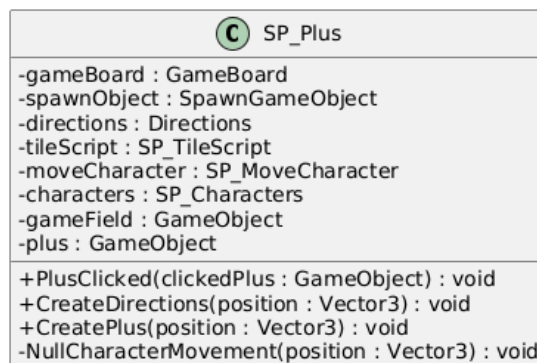
3.11. ábra. Eseményszoba

- **SP_MonsterTile** - szörnyszoba, létrehoz egy Monster objektumot és harcot indít (lásd 3.12. ábra)



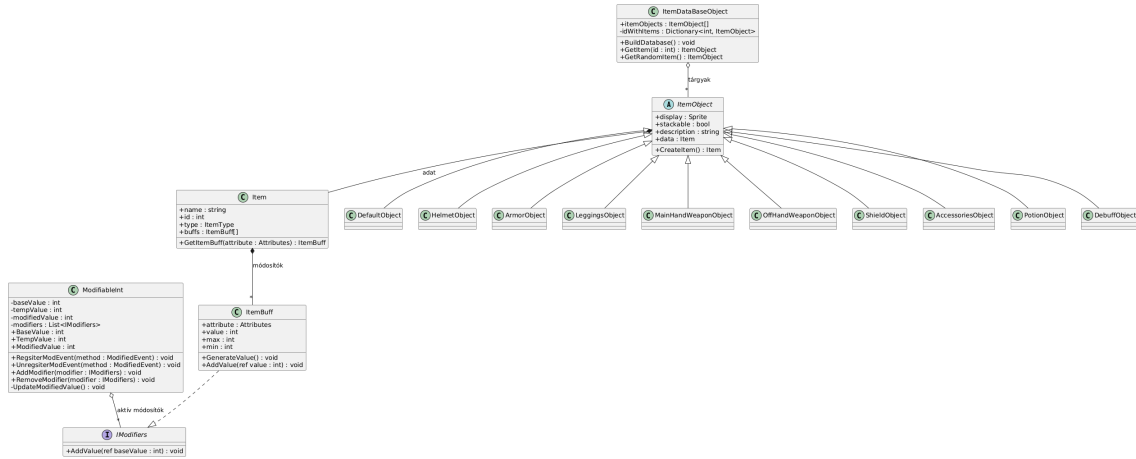
3.12. ábra. Szörnyszoba

-
- **Monster** - a szörnyes szobához tartozó ellenfél, nehézsége (*dangerLevel*) és jutalma véletlenszerűen generálódik az aktív karakter ereje alapján.
- **SP_Plus** – a játéktér bővítési pontja, új mezőt hoz létre kattintásra (lásd 3.13. ábra).



3.13. ábra. Plusz

- *PlusClicked*: új mezőt hoz létre a kiválasztott plusz helyén
- *CreateDirections*: létrehozza az új mező melletti lehetséges plusz objektumokat
- *CreatePlus*: létrehoz egy plusz objektumot a megadott pozíción
- *NullCharacterMovement*: bizonyos mezőtípusok esetén lenullázza az aktív karakter mozgáspontját



3.14. ábra. Tárgyrendszer szerkezete

3.6.2. Többjátékos játéktér elemek

A többjátékos mezők (MP_Tile, MP_Plus, MP_EventTile, MP_MonsterTile stb.) alapvetően azonos logikát valósítanak meg, mint egyjátékos megfelelőik, azonban:

- NetworkVariable formában tárolódnak
 - Ajtók (*NorthDoor, EastDoor, SouthDoor, WestDoor*)
 - Pozíció (*Position*)
 - Típus (*Class, Race, Attitude*)
 - Mozgásköltség (*Movement*)
- A mező beállítása kizárólag szerveroldalon történhet
- A kattintás kezelése RPC híváson keresztül továbbítódik a szerver felé
- Az objektumok Spawn/Despawn hívásokkal kezelhetők
- A harci jelenet és a harc eredménye hálózaton szinkronizált

3.7. Tárgyrendszer

A tárgyrendszer feladata a játékban használható tárgyak, azok típusainak és hatásainak kezelése. A tárgyak ScriptableObject alapú objektumokként vannak létrehozva, így Unity Editorban külön assetként szerkeszthetők (lásd 3.14. ábra).

3.7.1. Főbb osztályok:

- **ItemObject** - absztrakt alaposztály, tárolja:

- Megjelenített képet (*display*)
- Halmazhatóságot (*stackable*)
- Leírást (*description*)
- Item objektumot(*data*)

Leszármazottjai:

- HelmetObject
- ArmorObject
- LeggingsObject
- MainHandWeaponObject
- OffHandWeaponObject
- ShieldObject
- AccessoriesObject
- PotionObject
- DebuffObject

- **Item** - egy konkrét tárgy adatai:

- Név (*name*)
- Azonosító (*id*)
- Típus (*type*)
- Buffok tömbje (*buffs*)

Üres tárgyként is létrehozható (azonosító: mínusz egy)

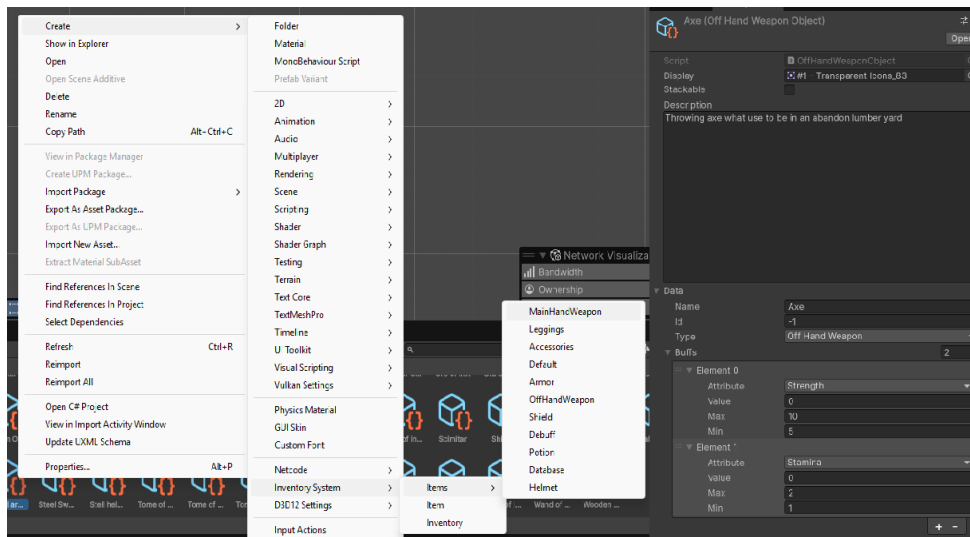
- **ItemBuff** - egy tárgy által adott attribútummódosító, értéke véletlenszerűen generálódik a megadott min–max között, az IModifiers interfészt valósítja meg.

- Típus (*type*)
- Érték (*value*)

- Maximum érték (*max*)
- Minimum érték (*min*)
- **IModifiers** - alap-, ideiglenes és módosítókkal számított értékkel rendelkező egész szám.
- **ItemDataBaseObject** - az összes tárgy adatbázisa, automatikus azonosító-rendelés, gyors keresés és véletlenszerű tárgyválasztás

3.7.2. Új tárgy hozzáadása a rendszerhez

A ScriptableObject alapú felépítés miatt új tárgyak hozzáadása egyszerűen elvégezhető a Unity Editorban: ki kell választani a megfelelő tárgytípust, meg kell adni a tulajdonságait, majd az adatbázishoz kell rendelni (lásd 3.15 ábra).



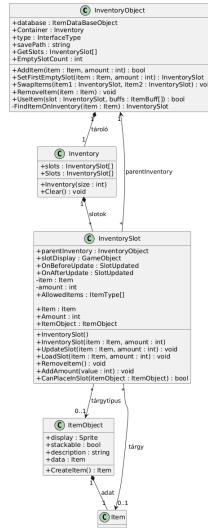
3.15. ábra. Tárgy létrehozása

3.8. Inventory rendszer

Az inventory rendszer feladata a karakterek tárgyainak és felszereléseinek tárolása, kezelése és módosítása. A rendszer három egymásra épülő osztályból áll.

3.8.1. Főbb osztályok:

- **InventoryObject** - ScriptableObject alapú inventory tároló, összeköti a slotokat a tárgyatbázissal (lásd 3.16 ábra).



3.16. ábra. Inventory felépítése

Feladat a tárgyak:

- Hozzáadásáért (*display*)
- Cseréjéért (*stackable*)
- Eltávolításáért (*description*)
- Aktiválásáért (*data*)

- **Inventory** - az inventory slotjait tároló osztály. Feladatai:

- Létrehozza a slot tömböt
- Szükség esetén kiüríti az inventoryt

- **InventorySlot** - egy konkrét slot leírása:

- szülőinventory (*parentInventory*)
- megjelenítő elem (*slotDisplay*)
- tárgy (*item*)
- mennyiség érték (*amount*)
- engedélyezett típusok (*AllowedItems*)

OnBeforeUpdate és OnAfterUpdate eseményekkel reagál a változásokra

Az inventory rendszer alapstruktúrájának kialakítása során a Coding With Unity oktatóvideó-sorozata[6] nyújtott inspirációt, azonban a végleges rendszer a projekt saját igényeihez igazodva jelentős mértékben továbbfejlesztve és testreszabva lett.

3.9. Felhasználói felület

A Roll for Loyalty felhasználói felülete hierarchikusan felépített menü- és játék-elemekből áll, amelyek biztosítják a különböző játékmódok kezelését, a karakteríranyítást és a játékállapot vizuális megjelenítését.

3.9.1. A főbb UI egységek:

- **MainMenu** – belépési pont, játékmód-választás
- **SinglePlayerMenu** / **MultiPlayerHostMenu** / **MultiplayerClientMenu** – mód-specifikus konfigurációs felületek
- **SinglePlayer UI** – inventory, equipment, vezérlőgombok, napló, statisztikák
- **MultiPlayer UI** – az egyjátékos UI kiegészítve chat rendszerrel és hálózati interakciókkal
- **Win képernyő** – a győztes nevének megjelenítése

3.9.2. A felhasználói felület logikai alapjai:

- **ToggleVisibility** – UI elemek megjelenítése vagy elrejtése
- **WriteLog** – események szöveges naplózása
- **DisplayInventory** – inventory vizualizáció
- **DynamicInterface** – slotok dinamikus létrehozása
- **ChangeCharacterSkin**: karakter kinézetének megváltoztatása

3.9.3. Támogatott interakciók:

- Kattintás és pointer események
- Drag and drop tárgykezelés
- Kameranavigáció
- Menü navigáció
- Chat kommunikáció (többjátékos mód)

3.10. Tesztelés

A tesztelés a Unity beépített Unity Test Framework segítségével történik, amely lehetővé teszi a program egyes részein elvégzett egységtesztelést (*unit testing*). A tesztek célja annak vizsgálata, hogy a játék főbb rendszerei

- Karakterkezelés
- Attribútumok
- Inventory
- Tárgyak
- Játéktér
- Harcrendszer

a várt módon működnek-e. A tesztelési folyamat különösen fontos a fejlesztési iterációk során: a Unity Test Framework biztosítja, hogy a módosítások után az alapvető funkciók továbbra is megfelelően működjenek, és az esetleges regressziók időben felismerhetők legyenek. A tesztek Edit Mode módban futnak, azaz nem igényelnek futó jelenetet (*scenet*), csupán a logikai komponensek izolált vizsgálatát végzik el. Minden tesztsztyában a *[SetUp]* attribútummal jelölt metódus hozza létre a szükséges objektumokat, a *[TearDown]* metódus pedig gondoskodik azok megsemmisítéséről, megelőzve az erőforrás-szivárgást.

3.10.1. Attribútum- és ModifiableInt-tesztek

A *ModifiableInt* osztály a játék attribútumrendszerének alapja: tárolja az alap- és módosított értéket, és kezeli a hozzáadott módosítókat. A tesztek ellenőrzik, hogy

- Alapértelmezett konstruktor nullával inicializál
- Alap- és az ideiglenes érték (*TempValue*) helyesen összegződik a módosított értékben
- A módosítók hozzáadása és eltávolítása a várt eredményt adja

Az *SP_Attribute*, *MP_Attribute* és *MP_Bot_Attribute* osztályok mindegyike egy *ModifiableInt* értéket tárol, és egy szülőobjektumhoz (*parent*) van kötve. A tesztek ellenőrzik, hogy a konstruktor helyesen állítja be az attribútum típusát (*Attributes* enum), az alapértéket és a szülőreferenciát, továbbá hogy több példány létrehozása esetén azok egymástól függetlenül kezelik saját értékeiket.

3.10.2. Játékos- és bottesztek

Az *SP_PlayerTests* és *SP_AttributeTests* osztályok az egyjátékos mód főszereplőjének, az *SP_Player* komponensnek a működését ellenőrzik. A tesztek lefedik a privát *GetClass* és *GetRace* metódusokat, amelyeket reflection segítségével hívnak meg:

- Érvényes bemenetre a megfelelő enum értéket várják vissza
- Érvénytelen bemenetre pedig azt, hogy az eredmény az enum definiált értékei közé essen

A *RandomClass* és *RandomRace* metódusok tesztelése 10–20 iteráción keresztül ellenőrzi, hogy a visszaadott érték minden esetben érvényes enum tag. A *SetReputation* metódus: helyes tömbméretét és nullával való inicializálását szintén ellenőrzik a tesztek, beleértve a nulla méretű tömb esetét is. Ezen kívül tesztelésre kerülnek

- *Level*
- *Active*
- *Fighting*

- *DrawableCards*
- *ReputationID*
- *Cheated*

property-k getter/setter párjai.

Az *SP_BotAttributeTests* és *MP_BotAttributeTests* osztályok az egyjátékos, illetve a többjátékos mód bot karaktereit tesztelik. Ezek a tesztek strukturálisan hasonlóak a játékos tesztekhez, ellenőrzik

- Attribútum-konstruktort
- Véletlenszerű osztály-, faj- és hozzáállás-generálást
- *RandomClass*
- *RandomRace*
- *RandomAttitude*
- Reputációs tömb helyes kezelését
- *Fighting*
- *Active*
- *Cheated*
- *WouldInterfier*

property-k alapértelmezett és beállított értékeit.

3.10.3. Tárgy- és inventory-tesztek

Az *ItemBuff* osztály tesztjei ellenőrzik:

- Konstruktor a megadott min-max tartományon belül generál véletlenszerű értéket
- Azonos min és max esetén az érték pontosan a megadott szám lesz
- Az *ItemBuff* *IModifiers* interfészen keresztül helyesen alkalmazható egy *ModifiableInt*-re

Az *Item* osztály tesztjei azt vizsgálják:

- Az alapértelmezett konstruktor üres nevet és mínusz egy-es azonosítót állít be
- az *ItemObject*-ből való konstruálás helyesen másolja:
 - Nevet
 - azonosítót
 - típust
 - buffokat
- A *GetItemBuff* metódus a helyes attributum alapján adja vissza a keresett buffot, vagy null-t, ha nem található.

Az *InventoryTests* osztály az *InventorySlot* és az *Inventory* osztályokat teszteli.

- Az *InventorySlot* esetén ellenőrzésre kerül
 - Az üres slot alapállapota (ID: mínusz egy, mennyiség: 0)
 - Az *UpdateSlot*
 - A *RemoveItem*
 - A *AddAmount*
- Az *Inventory* osztálytesztek azt vizsgálják
 - A konstruktor a megadott számú, egymástól független slotot hozza létre
 - A *Clear* metódus minden slotot visszaállít üres állapotba
 - Nulla méretű inventory létrehozása sem okoz hibát

3.10.4. Játéktér-tesztek

A *GameBoardTests* osztály a *GameBoard* komponens működését ellenőrzi. A tesztek vizsgálják:

- Az *addBoardPiece* metódus helyesen tárolja a pozíció–objektum párokat
- A *removeBoardPiece* hívás után az elem valóban eltűnik a tárolóból
- A nem létező kulcs eltávolítása nem dob kivételt

Az *onPosition* metódus tesztjei ellenőrzik:

- Üres táblán hamis, meglévő pozícióra igaz, eltávolítás után ismét hamis értéket ad vissza
- A *GetGameObject* metódus nem létező pozícióra null értékkel tér vissza
- A *GetRandomTilePosition* metódus tesztje 10 iteráción keresztül ellenőrzi, hogy a visszaadott pozíción mindig egy "Tile" tagú objektum található

3.10.5. Monster-tesztek

A *MonsterTests_EditMode* osztály ellenőrzi *Monster* osztály:

- Konstruktort
- Attribútumait
- Különböző támadóerő-értékkel a konstruktor kivétel nélkül fut le
- A generált *AttackPower* értéke az ellenség alaptámadó erejéhez képest ± 15 -ös sávon belül esik
- A *Reward* értéke minden esetben 1 és 4 közé esik

3.10.6. A tesztek összefoglalása

Az elkészített tesztek összesen több mint nyolcvan tesztet fednek le, amelyek a játék valamennyi kritikus alrendszerét ellenőrzik. A tesztek kialakítása során törekvés volt az izolált, egymástól független tesztetek készítésére, a határesetek (nulla méretű tömbök, érvénytelen bemenetek, negatív értékek) explicit vizsgálatára, valamint arra, hogy a véletlenszerű elemeket tartalmazó metódusok (pl. *RandomClass*, *AttackPower*) is elegendő iteráción keresztül kerüljenek ellenőrzésre. Ez a tesztelési alap lehetővé teszi, hogy a jövőbeli fejlesztések során a regressziók gyorsan azonosíthatók és javíthatók legyenek.

3.11. Továbbfejlesztési lehetőségek

A Roll for Loyalty jelenlegi változata egy teljes értékű, működőképes játékot valósít meg, azonban az architektúra tudatos tervezésének köszönhetően számos irányban tovább bővíthető.

Tartalom bővítése :

A játék legkönnyebben bővíthető területe a játéktartalom mennyiségi növelése. A ScriptableObject alapú tárgyrendszer és a moduláris pályaarchitektúra minimális fejlesztési erőfeszítéssel teszi lehetővé új tárgyak, szobatípusok, fajok és kasztok hozzáadását. A jelenlegi rendszer jó alapot biztosít ahhoz, hogy a kasztok és fajok ne csupán vizuális különbséget jelentsenek, hanem konkrét játékmechanikai hatásokkal is rendelkezzenek – például passzív bónuszokkal, egyedi képességekkel vagy hátrányokkal, amelyek stratégiai szempontból befolyásolják a karakter fejlődési útját.

Eseményrendszer kibővítése :

Az eseményszobák jelenleg véletlenszerű, előre meghatározott hatások közül választanak. A rendszer kiterjesztésével bevezethető lenne egy kiterjedtebb eseménytár, amely összetettebb, többlépéses eseményeket, feltételes kimeneteket és ritka, különleges hatásokat tartalmaz. Ez tovább növelné a játék kiszámíthatatlanságát és az újrajátszhatóság értékét.

Tárgyinterakciók gazdagítása :

A tárgyrendszer jelenlegi formájában passzív bónuszokat és egyszeri aktiválható hatásokat támogat. Lehetséges fejlesztési irány az aktív tárgyhasználat bővítése: kombinálható tárgyak, egymással szinergiában működő felszerelések, körönként aktiválható képességek, vagy akár tárgyak átadása más játékosoknak. Ezek az interakciók mélyebb stratégiai réteget adnának a játéknak.

Botokkal való interakció :

A mesterséges intelligencia jelenlegi reputációs rendszere alapot ad egy gazdagabb interakciós rendszer kiépítéséhez. Lehetséges fejlesztés lenne tárgycsere, szövetség-kötés, közös harc vagy egyéb kooperatív akciók bevezetése a bot karakterekkel. Ez a játék kooperatív dimenzióját erősítené, és dinamikusabb kapcsolatrendszert teremtené a játékosok és a mesterséges intelligencia között.

Csalási mechanika bővítése :

A jelenlegi csalási rendszer egységes szabályok szerint működik. Fejlesztési lehetőség több, egymástól eltérő csalási forma bevezetése, amelyek különböző kockázatokkal és jutalmakkal járnak, valamint finomabb büntetési fokozatok alkalmazása – például figyelmeztetési rendszer vagy progresszív szankciók.

Játékbeállítások kiterjesztése :

A jelenlegi beállítási lehetőségek viszonylag szűk körűek. Egy terjedelmes beállítási rendszer lehetővé tenné a pálya méretének, a győzelmi feltételek, a szörnyök nehézségének, a kezdő tárgyak mennyiségének, vagy akár egyedi szabályok konfigurálását. Ez a játékot rugalmasabbá tenné és különböző játékstílusokhoz igazíthatóvá.

4. fejezet

Összegzés

A Roll for Loyalty a Munchkin és a Betrayal at House on the Hill társasjátékok által inspirált, saját szabályrendszerrel rendelkező digitális játék, amely a karakterfejlődés, a stratégiai döntéshozatal, a felfedezés és a versengés elemeit ötvözi. A projekt célja egy olyan egyedi játékelmény megvalósítása volt, amely a klasszikus társasjátékok legizgalmasabb mechanikáit a modern videojáték-fejlesztés lehetőségeivel kombinálja. A fejlesztés során sikerült megvalósítani egy modulárisan bővülő játéktér-rendszert, összetett karakter- és tárgyrendszert, mesterséges intelligenciával támogatott ellenfeleket, valamint egyjátékos és többjátékos módokat egyaránt. A Unity játékmotor és a C# nyelv alkalmazása lehetővé tette, hogy a különböző rendszerek egységes, jól bővíthető architektúrában valósuljanak meg. Az egyjátékos és a többjátékos mód közötti legfontosabb különbség az adatkezelés és a kommunikáció szintjén mutatkozik meg. Egyjátékos módban minden logika lokálisan fut; többjátékos módban a Unity Netcode for GameObjects keretrendszer biztosítja a szinkronizációt NetworkVariable, NetworkList és RPC hívások segítségével. Ez az architektúra lehetővé teszi a két mód párhuzamos karbantartását és bővíthetőségét. A projekt megvalósítása során szerzett tapasztalatok alapján több fejlesztési irány is azonosítható a jövőre nézve. A játékbeli tartalom bővítése – további szobatípusok, karakterfajok, kasztok és tárgyak – viszonylag kevés fejlesztési erőfeszítéssel elvégezhető a ScriptableObject alapú tárgyrendszer és a moduláris pályaaarchitektúra miatt. Az online multiplayer támogatás bevezetése a Unity Relay és Lobby szolgáltatásainak integrálásával a helyi hálózaton túli játékot is lehetővé tenné. A mesterséges intelligencia tovább finomítható, például gépi tanulási módszerek bevonásával az adaptív döntéshozatal fejlesztésére. A Roll for Loyalty nem csupán játékfejlesztési

projekt, hanem komplex szoftverfejlesztési feladat is volt, amelynek során a tervezés, az implementáció, a felhasználói felület kialakítása, a hálózati működés és a tesztelés egyaránt fontos szerepet kapott. A projekt eredményeként egy működőképes, továbbfejleszthető alkalmazás jött létre, amely megmutatja, hogyan formálható egy kreatív ötlet részletes tervezéssel és kitartó fejlesztéssel teljes értékű programmá.

Irodalomjegyzék

- [1] Steve Jackson Games. *Munchkin Official Website*. URL: <https://munchkin.game/>.
- [2] Avalon Hill. *Betrayal at House on the Hill Official Website*. URL: <https://www.avalonhill.com/en-us/product/betrayal-at-house-on-the-hill-3rd-edition>.
- [3] Unity Technologies. *Unity Official Website*. URL: <https://unity.com/>.
- [4] Unity Technologies. *Netcode for GameObjects Documentation*. URL: <https://docs.unity3d.com/Packages/com.unity.netcode.gameobjects@latest/>.
- [5] Wikipedia contributors. *Dijkstra's algorithm*. URL: https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm.
- [6] Andy W. *Coding With Unity YouTube Channel*. URL: <https://www.youtube.com/@CodingWithUnity>.

Ábrák jegyzéke

2.1. Főmenü	6
2.2. Egyjátékos menü	7
2.3. Játékos testre szabása	7
2.4. Bot testre szabása	8
2.5. Speciális beállítások	8
2.6. Többjátékos menü	9
2.7. Többjátékos menü	10
2.8. Többjátékos Várakozó menü	10
2.9. Játéktér	11
2.10. Inventory	12
2.11. Equipment inventory	12
2.12. Statisztikák listája	12
2.13. Vezérlógombok	13
2.14. Eseménynapló	13
2.15. Kommunikációs felület	14
2.16. Csaló megjelölése	17
2.17. Tárgyak cseréje	19
2.18. Tárgy felszerelése	20
2.19. Tárgy törlése	20
2.20. Karakter és szörny harcolása	21
2.21. Nyertes képernyő	23
3.1. Az egyjátékos mód indításának folyamata	25
3.2. A többjátékos mód indításának folyamata	26
3.3. A játékos által végrehajtható műveletek	27
3.4. Egyjátékos karakter adatbázis szerkezete	28
3.5. Többjátékos karakter adatbázis szerkezete	31
3.6. Játéktér adatbázis szerkezete	32

3.7. Egyjátékos modell réteg szerkezete	33
3.8. Többjátékos modell réteg szerkezete	36
3.9. Játéktér elemei	37
3.10. Abstract szoba	37
3.11. Eseményszoba	37
3.12. Szörnyszoba	38
3.13. Plusz	38
3.14. Tárgyrendszer szerkezete	39
3.15. Tárgy létrehozása	41
3.16. Inventory felépítése	42